

ЕЛЕМЕНТИ РАЗВОЈА СОФТВЕРА

Начин полагања и критеријуми бодовања

Овај документ дефинише структуру полагања предмета, бодовање пројектног рада, контролне тачке, организацију одбране, теоријска питања, поправни термин и правила формирања коначне оцене.

71

максимум на пројектном делу

41

минимум пројектног дела

51

упис оцене без усменог

1. Структура полагања и формирање оцене

Провера знања на предмету заснива се на континуираном раду на рачунарским вежбама, изради и одбрани пројектног задатка и усменом делу испита, у складу са условима наведеним у наставку.

- Рачунарске вежбе и пројектни задатак: **највише 71 поен**.
- Минимум за успешно положен пројектни део: **41 поен**.
- Са **51 или више поена** на пројектном делу студент може да прихвати оцону формирану на основу остварених поена, **без полагања усменог дела испита**.
- Уколико студент који има **најмање 51 поен** са пројектног дела пријави испит, а не приступи усменом делу испита, **уписује се оцена која одговара броју поена остварених на пројекту**.
- Са **41 до 50 поена** на пројектном делу студент **приступа усменом делу испита**. За позитивну коначну оцону потребно је достићи **најмање 51 поен**.
- Резултат усменог дела може увећати укупан број поена за **највише 30**, односно умањити га за **највише 20 поена**.
- Коначан резултат исказује се на скали од **0 до 100 поена**. Резултат већи од 100 евидентира се као **100 поена**.
- Поени остварени на пројектном делу важе **до краја текуће академске године и не преносе се у наредну академску годину!**

Пројектни део	Минимум пројекта	Упис без усменог	Усмени део
до 71 поен	41 поен	од 51 поена	-20 до +30

2. Бодовање пројектног задатка

Пројектни задатак се вреднује по тематским целинама и контролним тачкама. Контролне тачке су саставни део укупног пројектног резултата и **не додају се преко максимума од 71 поена**.

Област	Поени
Tapiz Boards, Agile и Scrum - организација и праћење развојног процеса	10
Контролна тачка 1 - проблем, backlog и развојни ток	3
SOLID принципи и Clean Architecture	16
Контролна тачка 2 - архитектура и функционално језгро	3
Пословна логика и аутоматизовано тестирање	5
Контролна тачка 3 - тестирање и стабилна верзија функционалног језгра	6

Област	Поени
Интероперабилност и усклађеност уговорних страна	10
Правилна и контролисана употреба AI алата у изради пројекта	10
Контролна тачка 4 - AI/агентски развојни ток и завршна одбрана	8
УКУПНО	71

2.1. Додатни поени

За активност и ангажовање на настави могуће је остварити **до 5 додатних поена**. Додатни поени служе искључиво као допуна пројектног резултата до **максималних 71 поена** и **не могу повећати пројектни резултат преко тог прага**.

3. Контролне тачке

Током семестра реализују се **четири контролне тачке**. Њихова сврха је провера континуитета рада, стања пројекта и разумевања решења у одговарајућој фази развоја.

Контролна тачка	Област провере	Поени
КТ 1	Проблем, backlog и развојни ток	3
КТ 2	Архитектура и функционално језгро	3
КТ 3	Тестирање и стабилна верзија функционалног језгра	6
КТ 4	AI/агентски развојни ток и завршна одбрана	8

4. Присуство и обавезе на рачунарским вежбама

- **Присуство рачунарским вежбама је обавезно!**
- Студент који током семестра има **више од два неоправдана изостанка** са рачунарских вежби **губи право на полагање пројекта**.
- Да би изостанак био евидентиран као оправдан, одговарајућу потврду потребно је доставити у најкраћем могућем року на адресу danijel.jovanovic@uns.ac.rs.
- У поруци је потребно навести име и презиме, број индекса и датум вежбе на коју се изостанак односи, како би оправдање могло правилно да се евидентира.

5. Информације о пројекту

Пројектни задатак реализује се у оквиру заједничког софтверског производа **PYXIS** - информационог система за управљање дата центром. Детаљни функционални и технички захтеви дефинисани су актуелном пројектном спецификацијом, док су у наставку наведена правила организације рада која су обавезујућа за све тимове.

5.1. Тим и додела пројектне целине

- Тим чини **од 6 до 10 студената**.
- Сваком тиму предметни асистент додељује пројектну целину за коју тим преузима примарну одговорност у оквиру заједничког система.
- Сви тимови раде над **истим заједничким GitHub репозиторијумом**. Приступ репозиторијуму администрира предметни асистент.
- Приступ се додељује на основу GitHub корисничког имена. Студент је дужан да достави тачно корисничко име како би био додат као сарадник на репозиторијуму и како би његов индивидуални допринос био видљив кроз историју измена.

5.2. Начин рада у репозиторијуму

- Развојни ток обухвата Tapiz задатак, радну (feature) грану, смислене commit записе, Pull Request, CI проверу, преглед кода (code review) и завршну VERIFY/QA проверу пре означавања задатка као завршеног.
- **Директан push на main грану није дозвољен.** Значајне промене морају бити повезане са одговарајућим задатком и критеријумима прихватања, а Pull Request мора бити прегледан пре спајања.
- Када промена утиче на друге модуле или функционалност друге пројектне целине, неопходна је координација са тимом који ту целину одржава.
- Референтна технологија за израду је **.NET/C#**. Употреба друге технологије могућа је уз обезбеђену интероперабилност са остатком система.

5.3. Минимална очекивања од пројектне целине

- Пројектна целина **мора бити део заједничког производа** и имати јасно дефинисану границу и уговор према другим целинама; изолована мини-апликација која се не може интегрисати са системом не сматра се прихватљивом реализацијом.
- Очекују се аутоматизовани тестови кључних пословних правила и случајева употребе, као и интеграциони, уговорни (contract/cross-team) и регресиони тестови када су релевантни за додељену целину.
- Када домен то омогућава, током семестра очекује се **најмање једна значајна интеграција са другом пројектном целином**.
- Документација мора обухватити одговорност целине, главне случајеве употребе и критеријуме прихватања, јавне уговоре, значајне архитектонске одлуке (ADR), упутство за локално покретање и тестирање, репрезентативан демонстрациони сценарио и позната ограничења.
- Локално покретање, изградња (build) и тестирање не смеју зависити од недокументованих ручних корака познатих само једном члану тима.
- **Сваки студент мора имати видљив индивидуални допринос** и на одбрани бити способан да образложи сопствени рад, најмање један релевантан случај употребе, донету дизајнерску одлуку, припадајуће тестове и зависност према другој целини када она постоји.

6. Термини одбране пројекта

Одбрана пројекта организује се у **два основна термина**. Студент, односно тим, пројекат брани у једном од понуђених термина.

Термин	Период	Напомена
Предиспитни термин (предрок)	Прва половина децембра	Ранији термин за студенте чији је пројекат спреман за одбрану.
Редовни термин	Последња недеља зимског семестра	Редовни термин одбране пројекта.

Тачан датум, сатница и организација одбране биће благовремено објављени путем званичних канала комуникације предмета.

6.1. Одбрана пројекта

- На одбрани се проверава функционалност пројекта, испуњеност критеријума бодовања и разумевање донетих архитектонских, техничких и организационих одлука.
- Сваки члан тима мора бити у стању да образложи свој допринос и релевантне делове заједничког решења. **Присуство чланова тима чија се одбрана евидентира је обавезно.**
- Коришћење AI алата је дозвољено у складу са захтевима предмета, али **студент остаје одговоран за проверу, разумевање и образлагање сваког дела решења који представља на одбрани.**

6.2. Теоријска питања на одбрани

У оквиру одбране пројекта студент одговара и на теоријска питања која обухватају теме обрађене на предмету и њихову примену на реализованом пројекту. Питања су груписана по тематским областима у наставку.

6.2.1. Захтеви, backlog, Git и тимски развој

1. Објасните један User Story из вашег пројекта: која је улога, циљ и вредност коју доноси?
2. Како изгледа добар критеријум прихватања и по чему се разликује од нејасног описа захтева?
3. Покажите једну ставку backlog-а и објасните кроз које је фазе прошла до статуса Done.
4. Зашто Git историја треба да буде следљив запис развоја, а не само механизам за чување кода?
5. Шта чини квалитетну commit поруку и по чему се разликује од поруке попут "update" или "fix"?
6. Објасните један pull request из вашег репозиторијума: шта решава, шта мења, како је верификован?
7. Да ли сте имали Git конфликт током развоја? Како сте одлучили које понашање треба да буде коначно?
8. Шта значи да је ставка backlog-а заиста завршена (Definition of Done) у вашем пројекту?

6.2.2. OOP, Clean Code, SOLID и Clean Architecture

1. Покажите пример интерфејса у вашем пројекту и објасните који уговор он дефинише.
2. Објасните принцип енкапсулације на конкретној класи - зашто стање није јавно изменљиво без контроле?
3. За једну класу у пројекту објасните која је њена једина одговорност (Single Responsibility Principle).
4. Дајте пример где је Open/Closed принцип примењен - како додајете ново понашање без измене постојећег кода?
5. Објасните Liskov Substitution Principle на примеру из вашег кода - да ли се свака имплементација понаша у складу са уговором?
6. Шта је Interface Segregation Principle и зашто широк интерфејс може бити проблем?
7. Објасните Dependency Inversion Principle - да ли ваш пословни слој зависи од апстракција или од конкретних техничких детаља?
8. Нацртајте/описите слојеве Clean Architecture у вашем пројекту и смер зависности између њих.
9. Шта је composition root и где се у вашем пројекту налази регистрација конкретних зависности?
10. Који део вашег пројекта би највише патио када би Domain слој непосредно зависио од базе података?

6.2.3. Пословна логика и use-case слој

1. Изаберите један use-case и објасните његов улаз, ток и могући исход (успех и неуспех).
2. Како ваш ентитет чува сопствене инваријанте и спречава неважеће стање?
3. Објасните разлику између пословног (очекиваног) неуспеха и техничке грешке у вашем систему.
4. Покажите пример где сте уместо bool/null/генеричког изузетка користили експлицитан резултат операције.
5. Које зависности ваш use-case сервис добија кроз Dependency Injection и зашто?
6. Објасните на ком нивоу се у вашем систему врши валидација улаза, а на ком пословних правила.
7. Која операција у вашем пројекту мора бити третирана као једна атомска пословна трансакција?
8. Коју архитектонску одлуку сте променили током развоја и зашто?

6.2.4. Тестабилни дизајн и аутоматизовано тестирање

1. Покажите један тест и објасните Arrange-Act-Assert структуру на њему.
2. Објасните разлику између позитивног, негативног и граничног тест сценарија у вашем пројекту.
3. Где и зашто сте користили Moq (или сличан алат) за изолацију зависности?
4. Зашто тестабилан дизајн (DI, уске границе) олакшава писање тестова без базе и мреже?
5. Који део вашег система је тестиран интеграционим, а не само јединичним тестом, и зашто?
6. Шта покривеност кода (code coverage) говори, а шта не говори о квалитету тестова?
7. Да ли сте неки bug прво репродуковали тестом, а затим исправили? Опишите тај случај.
8. Шта је Git tag manual-core-baseline у вашем пројекту и зашто је тај тренутак важан?

6.2.5. Интеграција модула, уговори и подаци

1. Које модуле има ваш пројекат и која је пословна граница сваког од њих?
2. Који модул поседује који податак, и како други модули том податку приступају?

3. Објасните уговор између два модула - шта клијент зна, а шта не зна о другој страни?
4. Када у вашем систему користите синхрони позив, а када пословни догађај (event)? Зашто баш тако?
5. Дајте пример пословног догађаја из вашег система и објасните ко га објављује, а ко реагује на њега.
6. Како ваш систем решава конкурентан приступ истом ресурсу од стране два корисника или процеса?
7. Шта значи идемпотентност и где је у вашем пројекту потребна (нпр. поновљен захтев)?
8. Који интеграциони или уговорни тест штити границу између два ваша модула?

6.2.6. Контролисан развој уз AI подршку

1. Које пројектне инструкције (нпр. AI_INSTRUCTIONS.md) сте дефинисали и зашто баш те?
2. Покажите један запис из AI_USAGE.md и објасните која је одлука донета и како је верификована.
3. Коју сугестију AI алата сте прихватили, а коју одбацили? Образложите оба случаја.
4. Да ли сте направили сопствену процедуру (skill) за неки поновљиви задатак? Шта она ради?
5. Ако сте користили више агентских улога, која је одговорност и ограничење сваке од њих?
6. Зашто резултат AI алата сам по себи није довољан доказ да је промена исправна?
7. Када бисте у развоју вашег пројекта изабрали једноставан ток (један agent) уместо више улога?

6.2.7. MCP и повезивање са пројектом

1. Да ли ваш пројекат има MCP server? Које ресурсе или алате он излаже?
2. Објасните разлику између MCP ресурса и MCP алата на конкретном примеру из вашег пројекта.
3. Који стварни развојни сигнал (test, diff, status) враћа неки од ваших MCP алата?
4. Како сте применили принцип најмањих привилегија при дефинисању дозвољених операција server-а?
5. Шта је prompt injection и како бисте (или јесте) заштитили агентски ток од непоузданог садржаја?
6. Који податак или тајну ваш MCP server намерно не излаже и зашто?

6.2.8. Hooks, guardrails, евалуације и завршна провера квалитета

1. Коју детерминистичку проверу (hook) има ваш пројекат и на који догађај се она окида?
2. Објасните разлику између инструкције (препоруче моделу) и hook-а (извршивог механизма).
3. Који guardrail спречава ризичну операцију у вашем пројекту, и шта би се десило без њега?
4. Покажите један евалуациони сценарио (укључујући негативни) који проверава агентско понашање.
5. Ко је урадио реег QA преглед вашег пројекта и који је конкретан налаз или потврда забележена?
6. Који део вашег развојног тока мора бити гарантован кодом, а не само написан као инструкција?
7. Који је највећи преостали ризик или ограничење вашег пројекта и зашто сте га прихватили таквог?

6.2.9. Општа и интегративна питања

1. Проведите нас кроз цео пут једне функционалности: од захтева, преко кода и теста, до AI подршке ако је коришћена.
2. Која одлука у пројекту је била најтежа и зашто сте је донели баш тако?
3. Шта бисте урадили другачије да сада поново крећете са пројектом?
4. Који део система не разумете у потпуности и зашто (искрено, не као трик питање)?
5. Објасните једну аналогију између класичног принципа софтверског инжењерства и његове примене у развоју уз AI (нпр. интерфејс ↔ уговор алата, SRP ↔ агентска улога).

7. Поправни термин

У поправном термину студент израђује **нови пројекат самостално**. Пројектни поени из претходне реализације **не преносе се на нови пројекат**.

Број поена остварен на новом пројекту множи се **коефицијентом 0,8**. На тај начин максималан број поена који је могуће остварити на пројектном делу у поправном термину износи **56,8 поена**. На тако добијен резултат примењују се општа правила о минимуму пројектног дела, упису оцено и усменом делу испита.

8. Усмени део испита и коначна оцена

Усмени део испита обухвата проверу разумевања принципа обрађених на настави, архитектуре и имплементације пројекта, тестирања, интеграционих одлука и начина употребе AI алата.

- Резултат усменог дела може увећати укупан број поена за **највише 30**, односно умањити га за **највише 20 поена**.
- За студента који са **51 или више пројектних поена** приступи усменом делу, резултат усменог дела **улази у коначан обрачун поена**.
- Предмет је положен када коначан број поена износи **најмање 51**.

Оцена	6	7	8	9	10
Бодови	51-60	61-70	71-80	81-90	91-100

9. Одговорност за рад

Од студената се очекује јасно разграничење сопственог рада, рада осталих чланова тима и садржаја добијеног употребом спољних алата. У случају неовлашћеног преузимања туђег решења, плагијата или лажног представљања доприноса, за пројектни задатак евидентира се **0 поена**.

Употреба AI алата не умањује обавезу студента да разуме и самостално образложи решење. На одбрани се као релевантан доказ рада могу користити репозиторијум, историја измена, тестови, документација и евиденција употребе AI алата.

10. Комуникација и организационе информације

- Организационе информације, тачни термини контролних тачака и одбрана, као и евентуалне измене распореда, објављују се путем званичних канала комуникације предмета.
- За достављање потврде о оправданом изостанку користи се адреса danijel.jovanovic@uns.ac.rs.